Supplementary Material:
GPU-Scale Experiments on Random 3-SAT

Nadim Faris Nadim Zaro
Independent Researcher
zaronadim@gmail.com

September 3, 2026

Contents

1 Extended Methodology

1.1 Detailed GPU Implementation

Hardware Specifications:

- GPU: NVIDIA GeForce RTX 5070 Laptop GPU
- Compute Capability: 12.0
- Multiprocessors: 36
- CUDA Cores: 4,608 (estimated)
- Memory: 8.5 GB GDDR7

- Memory Bandwidth: 448 GB/s
- Peak FP32 Performance: 15 TFLOPS

Software Stack:

- CUDA: 12.x
- CuPy: 13.x
- Python: 3.13
- NumPy: 1.26
- Operating System: Windows 11

1.2 SAT Enumerator Optimization Details

Memory Management:

```
# Chunk size selection based on GPU memory
if n_vars <= 26:
    chunk_size = 2**26  # 67M assignments
elif n_vars <= 28:
    chunk_size = 2**25  # 33.5M assignments
else:
    chunk_size = 2**24  # 16.8M assignments
```

The reduced chunk size for $n \geq 29$, together with increased memory pressure, is the most plausible cause of the throughput drop observed at $n = 30$ in the main paper (Experiment 1).

Parallelization Strategy:

- Each GPU thread evaluates one clause across all assignments
- Clause results combined via parallel AND reduction
- Memory coalescing for optimal bandwidth
- Shared memory for clause data

Throughput Optimization:

1. Bit-manipulation for assignment generation
2. Vectorized clause evaluation
3. Early termination on first solution found (this makes per-instance "throughput" depend on where the first solution happens to lie)
4. GPU memory pooling to avoid allocation overhead

1.3 Sampled Solution-Graph Statistics

For each instance, up to 400–500 assignments are drawn uniformly at random; those satisfying the formula are retained. Distances are computed on GPU:

```
solutions_gpu = cp.array(solutions)  # Shape: (N, n)
diff = solutions_gpu[:, cp.newaxis, :] -
       solutions_gpu[cp.newaxis, :, :]
distances = cp.sum(cp.abs(diff), axis=2)
```

A graph is built at the *median* pairwise distance threshold:

$$L = D - A \quad \text{where} \quad A[i,j] = \mathbb{1}[d(i,j) \le t],$$

and we report

$$\beta_0 = \#\{\lambda(L) : \lambda \approx 0\} \quad \text{(connected components)} \tag{1}$$

$$\beta_1 = |E| - |V| + \beta_0 \quad \text{(graph cycle rank)}. \tag{2}$$

Interpretation notes: this is the cycle rank of one graph at one data-dependent threshold (no filtration is computed, so it is not persistent homology). Because the threshold is the median distance, about half of all vertex pairs are edges by construction, so β_1 grows roughly quadratically with the number of retained solutions — which is governed by clause density. The large cross-density differences in β_1 reported below are therefore primarily a solution-density and sample-size effect.

1.4 Instance Feature Extraction

128-dimensional feature vector $\phi(I)$ for SAT instance I:

Basic Features (16 dimensions):

- n (variables)
- m (clauses)
- m/n (clause-to-variable ratio)
- Clause length statistics (mean, std, min, max)
- Variable degree statistics

Graph Features (32 dimensions):

- Variable graph degree centrality
- Clause graph clustering coefficient
- Community structure metrics
- Graph diameter and radius

Structural Hashes (80 dimensions):

- Hash of clause patterns

- Symmetry group size
- Backbone variables (forced assignments)
- Resolution complexity estimates

The pairwise quantity reported in the main paper is $R(P_1, P_2) = \|\phi(P_1) - \phi(P_2)\|_2$: a dissimilarity between instance encodings, used as a descriptive statistic of the instance distribution.

2 Complete Experimental Results

2.1 Full SAT Enumeration Table

Table 1: Complete Extreme-Scale SAT Results

n	Assignments	Time (s)	Throughput (M/s)	SAT?	Solutions	Density
26	67,108,864	2.05	32.7	No	0	4.2n
27	134,217,728	1.06	126.2	Yes	1	3.0n
28	268,435,456	1.07	250.5	Yes	8	2.0n
29	536,870,912	0.55	978.8	Yes	2	4.267n
30	1,073,741,824	7.61	141.1	Yes	1	4.2n
Total	**2,080,374,784**	**12.34**	-	-	-	-

Satisfiable instances terminate at the first solution found, so the time and throughput columns mix hardware behavior with instance luck; the $n = 29$ run terminated after scanning roughly half its space, while the $n = 30$ run scanned most of it.

Additional Statistics:

- Average throughput: 168.6 M/s
- Peak throughput: 978.8 M/s (n=29)
- Satisfiability rate: 80% (4/5)
- GPU utilization: 87–94%
- Memory usage: 6.2–7.8 GB

2.2 Solution-Graph Statistics — Extended Results

The monotone trend — β_1 collapses as density rises — is what the sampling procedure predicts: at density 2.0 many uniformly random assignments satisfy the formula (large sampled graphs, quadratically many edges at the median threshold), while at density ≥ 4.2 almost none do (graphs with < 3 vertices are assigned $\beta_1 = 0$).

Statistical Tests:

- Kruskal-Wallis H-test: $H = 1847.2$, $p < 10^{-400}$
- Post-hoc pairwise comparisons (Dunn's test):
  - 2.0n vs. 4.2n: $p < 10^{-300}$

Table 2: Cycle rank β_1 by clause density (3000 instances). Labels in parentheses are the category tags used in the code and result files; all five generators draw uniformly random 3-clauses and differ only in density.

Density (code label)	n	Mean β_1	Std β_1	Max β_1	Instances
2.0n ("algebraic")	600	126.72	411.24	5,512	600
3.0n ("hierarchical")	600	8.92	40.38	443	600
4.267n ("phase_transition")	600	0.39	2.74	44	600
5.0n ("planted")	600	0.35	4.19	82	600
4.2n ("random")	600	1.26	7.49	83	600

– 2.0n vs. 3.0n: $p < 10^{-200}$

– All other pairs: $p < 0.001$

These tests confirm that clause density strongly affects the sampled statistic.

2.3 Feature-Distance Matrix — Full Statistics

Table 3: Feature-space distance distribution

Statistic	Value
Instance pairs analyzed	441,000
Matrix shape	441×1000
Computation time	1.17 seconds
Throughput	377,218 pairs/sec
Minimum	4.0
1st Quartile (Q1)	18.2
Median (Q2)	25.0
Mean	31.0
3rd Quartile (Q3)	42.8
Maximum	110.5
Interquartile Range (IQR)	24.6
Standard Deviation	18.7
Coefficient of Variation	0.60

2.4 Neural Network — Training Details

Task: binary classification of an instance-size category: label 0 = generated instances with ≤ 15 variables (one dataset slice), label 1 = instances of arbitrary size (another slice). All instances in both classes are random 3-SAT.

Architecture:

```
Layer 1: Dense(128 -> 512) + ReLU + Dropout(0.3)
Layer 2: Dense(512 -> 256) + ReLU + Dropout(0.3)
Layer 3: Dense(256 -> 128) + ReLU + Dropout(0.2)
```

`Layer 4: Dense(128 -> 2) + Softmax`

Training Hyperparameters:

- Optimizer: Adam($\beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$)
- Learning rate: 0.001 (fixed)
- Batch size: 512
- Epochs: 150
- Loss: Categorical cross-entropy
- Weight initialization: He normal

Training Progression:

Table 4: Neural Network Training Curve

Epoch	Training Loss	Best Loss
0	1.4143	1.4143
20	0.5440	0.5440
40	0.5482	0.5414
60	0.5389	0.5320
80	0.5388	0.5320
100	0.5361	0.5262
120	0.5259	0.5259
140	0.5267	0.5220

Confusion Matrix (1000 test instances):

		Predicted	
		small	mixed
Actual	small	356	144
	mixed	78	422

Performance Metrics:

- Accuracy: 77.8%
- Precision: 0.79
- Recall: 0.76
- F1-Score: 0.77
- ROC-AUC: 0.84

Feature-importance analysis attributes the predictions primarily to variable count (0.83) and clause density (0.76), consistent with the task definition: the network predicts size, because size is the label.

3 Reproducibility

3.1 Code and Data Availability

All code and data are publicly available:

- **GitHub Repository:** `https://github.com/big-brain-zaru/PvsNP`
- **Zenodo Archive (DOI):** `https://doi.org/10.5281/zenodo.18451652`

3.2 System Requirements

- NVIDIA GPU with compute capability ≥ 7.0
- 8+ GB GPU memory
- CUDA 11.0+
- Python 3.9+
- CuPy, NumPy, SciPy

3.3 Running the Analysis

```
# Install dependencies
pip install cupy-cuda12x numpy scipy

# Clone repository
git clone https://github.com/big-brain-zaru/PvsNP
cd PvsNP

# Main analysis: generation, enumeration, solution-graph
# statistics, feature distances, classifier
python gpu_pnp_breakthrough.py

# Extreme-scale enumeration benchmarks (n = 26..30)
python gpu_formalization.py

# Aggregation of the JSON outputs
python ultimate_proof.py
```

Notes on script output: `gpu_formalization.py` additionally prints an "oracle relativization" section whose values are produced by a hardcoded simulation stub (fixed multipliers on a baseline constant), and `ultimate_proof.py` prints heuristic per-method verdict labels and an aggregate score. Neither of these outputs is a measurement, and neither is reported in the paper; the scripts are kept as-is so that the published JSON result files remain exactly reproducible.

Expected runtime:

- Main analysis: 20–25 minutes
- Extreme-scale benchmarks: 10–15 minutes
- Aggregation: 2–3 minutes
- Total: 40 minutes on RTX 5070

4 Conclusion

This supplement documents the implementation and complete measurements behind the main paper: the GPU enumeration pipeline and its memory management, the sampled solution-graph statistics and their density dependence, the feature extraction and distance statistics, and the classifier training details. The measurements stand as benchmarking and instance-structure data for random 3-SAT; their scope is discussed in the main paper.